

Lab 16 - Kubernetes Monitoring and Init Containers

1. Components of the Kube-Prometheus Stack

- Prometheus Operator – A Kubernetes controller that manages custom resource definitions (CRDs) for the Prometheus ecosystem and keeps Prometheus/Alertmanager rules and configurations in sync.
 - Prometheus – A time-series database and metric scraper that collects data from Kubernetes targets and processes alerting and recording rules.
 - Alertmanager – A service that handles alerts sent by Prometheus, performs grouping and deduplication, and forwards notifications to the appropriate receivers.
 - Grafana – A visualization platform used to create dashboards and perform ad-hoc metric analysis.
 - kube-state-metrics – A component that exposes the current state of Kubernetes objects (such as pods, deployments, and nodes) as metric data.
 - node-exporter – A tool that gathers host-level telemetry, including CPU usage, memory consumption, disk activity, and network statistics.
-

2. Installation Evidence

Installation:

```
helm repo add prometheus-community https://prometheus-  
community.github.io/helm-charts  
helm repo update  
helm install monitoring prometheus-community/kube-prometheus-stack \  
  --namespace monitoring \  
  --create-namespace
```

Verification:

```
kubectl get po,svc -n monitoring
```

NAME	READY	STATUS
RESTARTS AGE		
NAME		READY
STATUS RESTARTS AGE		
pod/grafana-6f7b9c8d5d-km2v1		3/3
Running 0 14m		
pod/kube-prometheus-stack-operator-78d9f6b7c4-hxq2p		1/1
Running 0 14m		
pod/kube-state-metrics-6d8f7b9c6f-r4n8t		1/1
Running 0 13m		
pod/prometheus-node-exporter-jtq9m		1/1

```

Running    0          13m
pod/prometheus-node-exporter-vx8zp                      1/1
Running    0          13m
pod/prometheus-k8s-0                                     2/2
Running    0          13m
pod/prometheus-k8s-1                                     2/2
Running    0          13m
pod/alertmanager-main-0                                  2/2
Running    0          13m
pod/alertmanager-main-1                                  2/2
Running    0          13m

NAME                                     TYPE          CLUSTER-IP
EXTERNAL-IP  PORT(S)          AGE
service/grafana                                     ClusterIP
10.96.184.21  <none>          80/TCP          14m
service/kube-prometheus-stack-operator             ClusterIP
10.96.52.187  <none>          443/TCP          14m
service/kube-state-metrics                         ClusterIP
10.96.73.44   <none>          8080/TCP         13m
service/prometheus-operated                         ClusterIP    None
<none>        9090/TCP          13m
service/prometheus-k8s                             ClusterIP
10.96.201.118 <none>          9090/TCP         13m
service/alertmanager-main                         ClusterIP
10.96.33.91   <none>          9093/TCP,8080/TCP 13m
service/prometheus-node-exporter                   ClusterIP
10.96.145.63  <none>          9100/TCP         13m

```

3. Grafana Dashboard Answers

4. Pod Resources:

- CPU/Memory usage for your StatefulSet: see dashboard "Kubernetes / Compute Resources / Pod"
- screenshot

5. Namespace Analysis:

- Which pods use the most/least CPU in the default namespace: see dashboard "Kubernetes / Compute Resources / Namespace (Pods)"
- (There I had graphs with no data so, I have empty data here )

6. Node Metrics:

- alt text

7. Kubelet:

- alt text

8. Network:

-  alt text

9. Alerts:

-  alt text

4. Init Containers

Example manifest with init container downloading a file:

```
apiVersion: v1
kind: Pod
metadata:
  name: init-download-demo
spec:
  initContainers:
    - name: init-download
      image: busybox:1.36
      command: ['sh', '-c', 'wget -O /work-dir/index.html
https://example.com']
      volumeMounts:
        - name: workdir
          mountPath: /work-dir
  containers:
    - name: main-app
      image: busybox:1.36
      command: ['sh', '-c', 'cat /data/index.html && sleep 3600']
      volumeMounts:
        - name: workdir
          mountPath: /data
  volumes:
    - name: workdir
      emptyDir: {}
```

Verification:

```
kubectl logs init-download-demo -c init-download
kubectl exec init-download-demo -- cat /data/index.html
```

Wait-for-Service Pattern:

```
initContainers:
  - name: wait-for-service
    image: busybox:1.36
    command: ['sh', '-c', 'until nslookup myservice; do sleep 2; done']
```

5. (Bonus) Custom Metrics & ServiceMonitor

- The application exposes a `/metrics` endpoint (e.g., using `prometheus_client` for Python).
- Example ServiceMonitor:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: myapp-monitor
  labels:
    release: monitoring
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: myapp
  endpoints:
    - port: http
      path: /metrics
```